Solving OBELIX with Asymmetric PPO, Privileged Critics, and an Observation-Only Switch Policy

CS780 Capstone Project

Abstract

This report describes the final submitted controller for the CS780 capstone problem *OBELIX: the Warehouse Robot*. The task is a partially observable sequential decision-making problem in which a mobile robot must find a box, make contact, attach to it, and push it to the goal boundary while minimizing the damage caused by large stuck penalties. The final submission is a two-branch controller: a wall-aware actor trained with asymmetric proximal policy optimization (PPO) using a privileged critic, and a no-wall difficulty-3 actor trained with high-throughput vectorized PPO. At deployment, the policy uses only the legal 18-bit observation and memory derived from past legal observations and past actions. No simulator state, reward, attachment flag, coordinates, or hidden variable is read at inference time.

The main local evaluation target was difficulty 3 with horizon $T = 2000$, using the weighted score

$$0.4R_{\text{no-wall}} + 0.6R_{\text{wall}}.$$

On ten local seeds using `evaluate.py`, the clean switch submission obtained

$$R_{\text{no-wall}} = 1820.4, \qquad R_{\text{wall}} = -1142.8, \qquad 0.4R_{\text{no-wall}} + 0.6R_{\text{wall}} = 42.48.$$

Later local overtraining and router/macro experiments produced higher local weighted scores, including a macro-assisted candidate above 500, but those results were much less robust on hidden/Codabench seeds. I therefore treat the project as both a policy optimization exercise and an empirical study of the gap between local-seed performance and general seed robustness. The central lesson from this project is that OBELIX is not merely a search problem. It is a conservative partially observable control problem with severe credit assignment, large downside risk for incorrect forward motion, and different optimal behaviors in wall and no-wall settings. The final design worked because it explicitly addressed all three issues: partial observability, asymmetric training, and mode-dependent behavior.

1 Introduction

The OBELIX task is a reinforcement learning problem in the standard sense: an agent interacts with an environment over time, chooses actions from observations, receives scalar rewards, and aims to maximize long-run return. Unlike supervised learning, correct actions are never provided directly. Instead, the controller must discover useful behavior through trial-and-error interaction and must balance exploration against the large cost of poor actions. This is especially important here because the task is not fully observable, the box can blink or move before contact, and obstacle walls can create states where a locally aggressive push policy is catastrophic.

This framing closely matches the early CS780 lectures on the agent-environment loop, evaluative feedback, and delayed consequences. OBELIX is a particularly severe example of that lecture material because the penalty for a bad action often arrives only after the robot has already committed to a bad wall or box geometry.

From the assignment description, the robot must first *find* the box, then *push* it after contact, and finally *unwedge* it at the boundary. These are human-interpretable phases, but they are not given directly to the agent. The policy must infer them from the 18-bit observation stream. This makes the problem closer to a POMDP than a fully observed MDP, and it explains why memory and history-dependent features matter.

The rest of this report first formalizes the task as a POMDP, then explains the legal observation space, the vectorized torch environment, and the reward geometry. After that, it tells the chronological story of the methods I tried over the project: DDQN, symmetric PPO, handcrafted controllers, asymmetric PPO, recurrent memory, switching, distillation, and wall-search experiments. The final sections then describe the selected architectures, evaluation, legality, and conclusions.

2 Formal Problem Definition

2.1 Environment Description

OBELIX is a bounded two-dimensional robot-control task. A circular robot moves in a square arena and must push a square box to the goal boundary. In the hardest setting used for the final submission, the box can blink on and off and can move before attachment. In the wall-obstacle version, a wall with a small opening can appear, forcing the robot to choose approach angles and pushing directions carefully.

The action set is discrete:

$$\mathcal{A} = \{\texttt{L45}, \texttt{L22}, \texttt{FW}, \texttt{R22}, \texttt{R45}\}.$$

The turn actions rotate the robot by 45° or 22.5°. The `FW` action attempts a forward move of 5 pixels. When the box is attached, forward motion attempts to translate both the robot and the box.

Episodes terminate either when the box is successfully removed or when the horizon T is reached. For the final evaluation, $T = 2000$.

2.2 POMDP Formulation

The interaction is naturally written as a partially observable Markov decision process (POMDP)

$$\mathcal{P} = (\mathcal{S}, \mathcal{A}, \Omega, P, O, r, \rho_0, \gamma, H).$$

Each component is defined below.

This formalization follows the course treatment of RL as an MDP together with practical extensions. In the lecture notes, the MDP tuple is the clean starting point; in this project the important complication is that the deployed policy does not observe the full state, so the actual control problem is partially observable even though the simulator itself is Markov.

Latent state space $\mathcal{S}$. The simulator state $s_t \in \mathcal{S}$ contains all variables required to make the dynamics Markov. In this task that includes at least the robot pose, robot heading, box pose, box velocity, box visibility/blinking state, attachment state, arena boundary geometry, wall geometry when obstacles are enabled, and the internal variables required by the simulator to resolve collisions and termination.

Action space $\mathcal{A}$. The agent chooses one of the five legal discrete actions at each time step.

Observation space Ω. The deployed agent does not observe s_t directly. Instead it receives a binary observation

$$o_t \in \Omega = \{0,1\}^{18}.$$

This is the legal sensor interface exposed by the environment.

Transition kernel P. The transition model

$$P(s_{t+1} \mid s_t, a_t)$$

contains the motion of the robot, the pre-attachment motion of the box, the blinking process, collision resolution, wall interactions, and the state update after contact or goal completion.

Observation model O. The observation process is

$$O(o_t \mid s_t).$$

This maps the latent simulator state to the legal 18-bit sensor vector. Because many different latent states can produce the same observation, the agent must infer task phase and hidden structure from observation histories rather than from a single frame.

Reward function r. The scalar reward is generated from the current transition. It includes a per-step cost, large stuck penalties, one-time sensor rewards, an attachment bonus, and a large success bonus when the box reaches the boundary.

Initial-state distribution ρ_0. At the beginning of each episode the latent state is sampled from a distribution over robot pose, box pose, box velocity, blinking mode, and wall configuration. This matters because the policy must work over many randomized starts rather than a single fixed layout.

Discount factor γ. The PPO training objective uses discounted return with $\gamma \in (0,1)$. For the final wall branch, $\gamma = 0.995$. This favors high reward sooner rather than later and is consistent with the capstone objective of solving the task quickly and safely.

Horizon H. The environment is episodic with finite horizon $H = T$. The final evaluation used $T = 2000$.

2.3 History-Dependent Policy

Because the observation is partial, the optimal action generally depends on history, not just the current observation. Formally, the deployed controller is not a memoryless map from o_t to a_t; it is a history-dependent policy

$$\pi(a_t \mid h_t), \qquad h_t = (o_0, a_0, o_1, a_1, \ldots, o_t).$$

In the actual implementation, the full history is compressed into finite legal memory features built from observation stacks, action history, masks, and counters. This is the correct way to make a POMDP manageable without violating the rules.

2.4 Task Phases as Latent Modes

Although the simulator does not expose symbolic stage labels, the task can be understood as three latent behavioral modes:

1. **Search / find:** the robot must rotate and move to bring the box into the sensor sectors.
2. **Approach / attach:** the robot must use local sensor geometry to make contact from a favorable side.
3. **Push / unwedge:** after contact, the robot must drive the box to the goal boundary while avoiding wall traps and repeated stuck penalties.

These modes are not given to the policy. They must be inferred from the observation stream. This is one of the main reasons plain reactive policies were unreliable.

3 Legal Observation Space

The raw observation is an 18-bit binary vector

$$o_t = [b_0, \ldots, b_{15}, b_{\mathrm{IR}}, b_{\mathrm{stuck}}].$$

The first 16 bits are sonar-like range bits. They are naturally grouped by direction:

$$\begin{aligned} \text{left}: &\quad b_0, \ldots, b_3, \\ \text{front}: &\quad b_4, \ldots, b_{11}, \\ \text{right}: &\quad b_{12}, \ldots, b_{15}. \end{aligned}$$

The final two bits are the infrared sensor and the stuck flag:

$$b_{\mathrm{IR}} = b_{16}, \qquad b_{\mathrm{stuck}} = b_{17}.$$

The infrared bit indicates that the grey box is directly ahead; the stuck bit indicates that the robot is jammed against a wall or boundary.

The submitted policy never reads the simulator state, reward, done flag, box coordinates, robot coordinates, attachment flag, or any other hidden variable. All inference-time memory is constructed only from legal observations and the policy's own past actions.

4 High-Throughput Environment: `obelix_torch.py`

The original simulator, `obelix.py`, is a NumPy/OpenCV-style environment designed for correctness, visualization, and assignment evaluation. It is the reference environment for `evaluate.py` and Codabench. However, plain PPO needs a very large number of rollouts. In this project, single-environment training was too slow and also too statistically noisy: one PPO update from a small number of episodes did not contain enough successful or near-successful trajectories to produce a useful gradient.

For that reason, I created `obelix_torch.py`. The goal was not to define a new task, but to port the same observation, reward, transition, wall, collision, blinking-box, and moving-box logic into a tensor-friendly implementation. The file contains two useful interfaces:

- `OBELIX`: a torch-based single environment with the same public API as `obelix.py`, useful for parity testing and debugging;
- `OBELIXVectorized`: a single-process batched environment that stores N independent OBELIX states in torch tensors and steps all environments together on the GPU.

The vectorized environment was essential for PPO. PPO alternates between rollout collection and minibatch optimization. If the rollout batch is small, the advantage estimates are dominated by seed variance and rare catastrophic stuck events. With `OBELIXVectorized`, I could run hundreds to thousands of environments per rollout on CUDA. A typical setting was

$$N \in \{256, 512, 768, 1024\}, \qquad L \in \{64, 128\},$$

where N is the number of parallel environments and L is the rollout length. A single PPO iteration could therefore collect $N \cdot L$ transitions, e.g. $512 \cdot 128 = 65{,}536$ transitions, without launching thousands of Python processes.

This mattered in four concrete ways.

1. **Throughput:** the policy, reward computation, wall collision checks, box motion, and observation generation could be evaluated in batched tensor form. This made long PPO sweeps practical on the RTX GPU.
2. **Diversity per update:** every PPO update saw many randomized starts. This reduced overfitting to a single unlucky trajectory and made the no-wall branch much stronger.
3. **Asymmetric PPO support:** the vectorized torch environment also exposed the privileged tensors needed by the critic, such as robot pose, box pose, relative geometry, wall gap geometry, push state, box velocity, and boundary margins.
4. **Fast ablations:** the same training code could sweep different observation stacks, action-history lengths, entropy coefficients, GRU/LSTM variants, reward shaping terms, and warm-start teachers.

The most important engineering requirement was parity. The torch environment had to match `obelix.py`; otherwise the policy would train on one task and be evaluated on another. During development I found and fixed parity issues, including a wall-obstacle sonar mismatch where one extra sonar bit could change the reward under the same state and action. After this, `obelix_torch.py` was treated as the training backend, while `obelix.py` and `evaluate.py` remained the evaluation authority.

5 Reward Dynamics and Credit Assignment

The reward function is unusually conservative. A stationary policy that avoids all useful contact still incurs the step cost, so over a horizon of T steps it can score roughly $-T$. At $T = 2000$, this means a passive agent is already close to -2000. A policy that repeatedly drives into a wall can be far worse because the stuck penalty is large and can occur many times.

The per-step reward implemented by `obelix.py` is summarized as

$$r_t = -1 - 200\,\mathbf{1}\{b_{\text{stuck}} = 1\} + r_{\text{sensor-once}} + r_{\text{attach}} + r_{\text{success}}.$$

The one-time sensor rewards are

left/right sonar bits	+1 per bit, once per episode,
front far bits	+2 per bit, once per episode,
front near bits	+3 per bit, once per episode,
IR bit	+5 once per episode.

First attachment gives +100, followed by the normal push-state step cost. Pushing the attached box to a boundary gives a success bonus of +2000.

This reward geometry creates a hard credit-assignment problem:

- **Search is necessary** because the box may be invisible, moving, or outside the sensing region.
- **Risk is asymmetric:** a single useful forward move may help only slightly, but a single bad forward move can produce a -201 transition.
- **Attachment is not enough:** contact from the wrong side often leads to a jammed push and repeated stuck penalties.
- **Myopic safety is misleading:** a policy that mostly rotates or hesitates can avoid catastrophic penalties, but still fail the task and accumulate a large negative return from the step cost.

This explains why many early agents looked active in rollouts but still scored poorly. The policy had to be exploratory enough to find the box, structured enough to infer task phase from partial observations, and conservative enough to avoid repeated collision loops.

This is exactly the kind of delayed-consequence setting emphasized in the early reward and environment lectures: a locally plausible move can have very poor long-run value, and the agent has to learn that from trajectories rather than from direct supervision.

6 Chronological Development Methodology

This section is the project story. The final agent was not designed in one step. Over approximately 1.5 months, I repeatedly trained a method, evaluated the actual behavior, identified the failure mode, and then changed the representation or algorithm. The sequence was: value-based DDQN, symmetric PPO, handwritten phase controllers, asymmetric PPO with privileged critics, recurrent memory and teacher transfer, explicit wall/no-wall switching, and finally difficulty-3 wall-search, router, macro, and distillation experiments.

6.1 First Attempt: Value-Based DDQN

The first learning attempt was DDQN because the action space is small and discrete. The idea was straightforward: learn a function

$$Q_\theta(h_t, a_t)$$

and choose the action with the highest estimated value. The implementation used a replay buffer, a target network, minibatch temporal-difference updates, and many parallel environment workers so that the replay buffer would contain more than one seed's trajectory distribution. I also tested GPU updates for the Q-network while the environment workers collected experience.

This approach helped establish a baseline, but it failed for a structural reason. The 18-bit observation is heavily aliased. The same local sonar pattern can mean "box is near and forward is

useful" or "wall is near and forward is dangerous". DDQN tried to assign one value to such aliased transitions, so the target estimates became unstable. The bootstrapped target

$$r_t + \gamma \max_a Q_{\theta^-}(h_{t+1}, a)$$

was especially noisy because a single stuck transition can dominate several harmless turn transitions. Replay did not solve the problem because replay gives more samples, not better state information. This motivated the move away from pure value-based learning toward PPO with explicit history features.

At a conceptual level, this stage followed the DQN, DDQN, and replay-buffer progression from Lectures 11–13. The main lesson from the OBELIX experiments was that those fixes were still not enough once the dominant issue became observation aliasing rather than just unstable optimization.

6.2 Second Attempt: Symmetric PPO

The next stage was symmetric PPO, where both actor and critic received the same legal observation representation. PPO was a better fit because it directly optimized the policy and used clipped updates to prevent the large policy jumps that I saw in early value-based experiments. The basic PPO loop was:

1. collect rollouts from hundreds of vectorized OBELIX environments;
2. compute value estimates and generalized advantage estimates;
3. update the actor with the clipped PPO objective;
4. update the critic on the same legal observation features;
5. repeat while sweeping entropy, learning rate, rollout length, and hidden sizes.

This phase made the no-wall problem much more tractable. The raw 18 bits were augmented with simple derived features: left/front/right counts, front-near and front-far counts, a blind flag, stuck and IR bits, and left-right/front-side imbalances. This did not add any illegal information; it only made useful sensor aggregates easier for the MLP to learn. On no-wall difficulty 3, this style of PPO eventually produced a very strong policy because the main problem was search and approach rather than wall navigation.

The same method did not solve wall obstacles. The wall setting required the critic to predict whether a short local observation history would eventually lead to a successful push, a harmless turn, or a repeated stuck loop. A legal-only critic was too weak for that. It often assigned optimistic value to states that looked locally promising but were geometrically bad, especially when the robot approached the wall or attached to the box from the wrong side. This was the main reason I moved to asymmetric PPO.

This was also the point where the project started to line up more with the policy-gradient and actor-critic half of the course. Once the representation itself became the main bottleneck, directly optimizing the policy was more productive than continuing to refine Q-targets.

6.3 Handwritten Controllers as Diagnostics

In parallel with PPO, I wrote several manually structured controllers. The goal was not only to submit a hand-coded agent; it was to understand what the learned agent needed to know.

These controllers used sector counts, last-seen side, blind-search direction, stuck recovery plans, and explicit phases:

$$\text{search} \rightarrow \text{approach} \rightarrow \text{attach} \rightarrow \text{push/unwedge}.$$

The useful lesson was that a single observation was insufficient. A good controller needed to know whether a front bit was newly seen or persistent, whether the last forward move produced stuck, whether the agent had been blind for many steps, and which side had last seen the object. I also tried phase-based controllers inspired by the human description of the task: one routine to find the box, one to center the box, and one to push or recover. These worked on some seeds but were brittle. A small change in box motion or wall geometry could turn a correct push into a repeated wall collision.

This stage directly informed the later learned representation. Instead of relying only on the current 18 bits, the actor would need action history, observation history, contact memory, blind counters, repeated-observation counters, and stuck memory.

6.4 Asymmetric PPO with Privileged Critic

The next major step was asymmetric PPO. The actor remained legal:

$$\pi_\theta(a_t \mid x_t),$$

where x_t was built from legal observations and the agent's own past actions. The critic received an additional privileged vector:

$$V_\psi(x_t, p_t).$$

This was used only during training. At inference, the actor did not receive p_t.

The privileged critic was given information that is useful for value estimation but illegal for deployment: robot position and heading, box position, relative robot-box vector, relative heading, push/attachment state, box visibility, stuck state, step fraction, difficulty, wall flag, moving-box velocity, boundary margins, distance to the goal boundary, and wall-gap geometry. The purpose was to make the critic understand whether a state was actually good or bad even when the legal observation was ambiguous.

This helped in two ways. First, the advantage estimates became less noisy because the critic could evaluate true geometry during training. Second, it allowed the actor to learn correlations between legal histories and hidden states. For example, the actor could not see the wall coordinates, but it could learn that a repeated stuck pattern after a forward action, combined with certain side-sonar histories, usually meant that the policy should turn away and avoid pushing forward again.

Asymmetric PPO still did not fully solve the wall environment, but it was a clear improvement over symmetric PPO and became the basis for the final wall branch.

The actor-critic logic here is very much in the spirit of Lecture 15: use a critic to reduce policy-gradient variance, and use better value estimates to produce better advantages. My extension was to make the critic privileged during training while keeping the actor legal at deployment.

6.5 Memory, Relative Pose, and Recurrent Policies

The next idea was to make the actor more stateful. I tried feed-forward history stacks, GRU policies, LSTM policies, and manually constructed recurrent features. The memory features included:

- observation stacks, to remember which sensor bits were persistent or newly seen;
- action history, to distinguish "front bit after turning" from "front bit after moving forward";

- legal dead-reckoned pose from the agent's own turns and successful forward moves;
- heading estimates from the executed turn actions;
- blind, stuck, contact, turn-streak, and forward-streak counters;
- wall-hit memory by heading bin, created only when `FW` led to the stuck bit;
- last-seen side, front-strength estimates, and sensor-seen masks.

The intended wall logic was: if a forward move causes stuck, remember the approximate relative position and heading as a wall-hit; if similar headings or nearby dead-reckoned positions repeatedly produce stuck, treat that as evidence of a long continuous wall rather than the box; then bias the policy away from that heading while continuing to search for box-like contact. This is legal because the pose estimate is internal dead reckoning, not simulator pose.

I also tried teacher transfer. In those runs, a privileged teacher or privileged critic helped generate better actions during warm-start or behavior cloning. This improved some training curves, but it introduced a mismatch: the student sometimes looked good when the teacher rescued difficult states, then collapsed when evaluated alone. The lesson was that teacher assistance had to be used carefully. A final submission cannot depend on teacher takeover; it must recover using only its own legal memory.

6.6 Switching Between Wall and No-Wall Policies

The strongest practical improvement came from separating the wall and no-wall policies. The no-wall policy learned search and push behavior well. The wall policy was more conservative and had memory features for avoiding repeated stuck states. A single monolithic policy often compromised badly: if it acted like the no-wall policy in wall episodes, it got punished; if it acted like the wall policy in no-wall episodes, it lost easy positive reward.

The switch policy used an early probe window. For roughly the first 80 steps, it ran the wall branch and accumulated legal statistics:

$$\text{blind count}, \quad \text{front/contact count}, \quad \text{total front sensor hits}.$$

After the probe, it routed to the no-wall PPO branch if the trajectory looked like no-wall: long blind segments, low contact, and no evidence of wall-like obstruction. Otherwise it stayed with the wall branch. This switch was not perfect, but it was a major improvement because it let the no-wall policy remain aggressive while keeping a more conservative wall branch for obstacle cases.

I then added exact-boost logic for a small number of early observation patterns that the neural wall branch handled poorly. The boost controller used only legal observations and dead-reckoned memory, but it was more hand-engineered than the neural branches. It turned away from side detections, committed forward when front/IR evidence was strong, and started recovery plans after stuck events.

6.7 Difficulty-3 Wall Search

The final phase focused on difficulty 3 with wall obstacles. At this point the no-wall branch was strong; the bottleneck was wall robustness. I tried several directions:

- continuing the deployed wall actor with more wall-only PPO updates;
- push-heavy shaping to encourage box progress after attachment;

- high-entropy/exploratory variants to escape local turn-heavy policies;
- recurrent GRU/LSTM asymmetric PPO with privileged critic and teacher losses;
- dead-reckoning variants with wall-hit memory and heading bins;
- cost-sensitive routers that penalized wall-to-no-wall misclassification more heavily than no-wall-to-wall misclassification;
- 80-step wall/no-wall detectors trained from probe trajectories;
- distillation from the best macro and switch policies into recurrent students.

The detector idea was promising but not sufficient. A scan probe could often tell wall from no-wall, but the scan itself damaged some no-wall episodes before the no-wall policy took over. A less disruptive spin probe was safer but missed important wall cases. This showed that wall/no-wall detection is not just a classification problem; the probe trajectory must also preserve reward.

The dead-reckoning wall policy also improved some internal wall-only evaluations. The logic was to infer wall structure from long continuous stuck/contact evidence, keep a relative memory of bad headings, and avoid repeating forward actions into the same wall. However, these wall-only policies were poor on no-wall episodes and therefore had to be used only behind a router. They also remained high variance across seeds.

The seed-to-seed instability is another practical instance of the bias-variance tradeoff discussed around actor-critic and GAE in the lectures. A method can look strong on a small seed slice and still be too high variance to trust as a submission.

6.8 Macro Policies, Distillation, and Local Overtraining

The highest local weighted scores came from macro-assisted and learned-router variants. The macro policy matched a small number of exact legal 18-bit observation signatures, took a short probe action when a start was ambiguous, and then played a fixed action sequence if the next observation matched a known signature. This did not access illegal runtime information: it used only the 18-bit observation and the episode reset key. But it was still hand-engineered trajectory knowledge extracted from local seed analysis.

I then tried to distill this behavior into a learned student. The student was trained on expert trajectories from the macro wall policy and the strong no-wall policy. The idea was to remove explicit macro tables while preserving behavior. This worked locally: learned-router variants could get very high weighted scores on seeds $0, \ldots, 9$. But Codabench and random-seed checks showed that this was still overtraining. The student had learned the local demonstration distribution, not a general wall-solving strategy.

The honest conclusion is that the 500+ local weighted reward was achievable, but it was not a deep solution to the general wall-obstacle problem. The more robust insight was the methodology: use high-throughput PPO for no-wall, asymmetric privileged critics for wall training, legal memory/dead reckoning for partial observability, and a cautious router because the wall and no-wall regimes require different behaviors.

6.9 Empirical Conclusion After the Full Search

I did not prove that difficulty-1 or difficulty-2 wall-obstacle variants are unsolvable. However, after the full experimental search, my empirical conclusion was that I did not find a robust general solution for wall obstacles. The reward dynamics explain why: doing nothing loses roughly the

horizon, but repeated wall contact can be much worse. The policy therefore has to be exploratory enough to find the box and conservative enough to avoid repeating a bad forward action. That balance was difficult to learn from 18 bits under moving/blinking box dynamics.

The best scoring strategy became: first avoid misclassifying the environment mode, then solve no-wall very well, and keep wall behavior non-catastrophic. This is why the final policy family is best described as a routed mixture of specialized policies rather than a single universal wall solver.

7 Observation Engineering

7.1 No-Wall PPO Encoding

For the no-wall branch, the best checkpoint used a compact encoded observation of dimension 32. It concatenates the raw 18-bit observation with 14 derived features:

$$\phi_{\text{nw}}(o_t) = [o_t, d_t] \in \mathbb{R}^{32},$$

where d_t contains

directional hit flags	$\mathbf{1}\{\text{left hit}\},\ \mathbf{1}\{\text{front hit}\},\ \mathbf{1}\{\text{right hit}\},$
terminal bits	$b_{\text{IR}},\ b_{\text{stuck}},\ \mathbf{1}\{\text{blind}\},$
counts	$n_{\text{left}},\ n_{\text{front}},\ n_{\text{right}},\ n_{\text{front-far}},\ n_{\text{front-near}},$
differences	$n_{\text{left}} - n_{\text{right}},\ n_{\text{right}} - n_{\text{left}},\ n_{\text{front}} - \frac{1}{2}(n_{\text{left}} + n_{\text{right}}).$

This representation made the relative salience of front versus side hits directly visible to a small multilayer perceptron and was sufficient in the no-wall difficulty-3 setting, where the main challenge was search rather than fine obstacle negotiation.

7.2 Wall Actor Memory Features

In the wall setting, a single observation was usually insufficient. The deployed wall actor uses the legal feature vector

$$x_t = [o_{t-11:t}, \text{onehot}(a_{t-6:t-1}), m_t, c_t],$$

where $o_{t-11:t}$ is a 12-frame observation stack, the action history stores the last 6 actions, $m_t \in \{0,1\}^{17}$ is a sensor-seen mask over bits $0 \ldots 16$, and $c_t \in \mathbb{R}^8$ contains normalized counters. These counters encode blind streak, stuck streak, last-seen side, contact memory, step fraction, repeated-observation streak, blind-turn streak, and stuck memory. For the final wall checkpoint,

$$\dim(x_t) = 12 \cdot 18 + 6 \cdot 5 + 17 + 8 = 271.$$

All of these features are legal because they are deterministic functions of current and past legal observations together with the actions already emitted by the policy.

8 Learning Method: Why PPO and Why Asymmetric Training

8.1 Why a Policy-Gradient Method Was Used

Several families of methods were plausible from the course material: tabular methods, Monte Carlo control, TD methods such as Q-learning, and policy-gradient methods. In practice, the deployed solution used PPO because the task combines partial observability, large downside risk, reward shaping during training, and function approximation.

A tabular method is not suitable here because the latent state is continuous and hidden. Even the legal observation history is too large to tabulate. Pure Monte Carlo updates are also unattractive because they use the full return only at episode end, which causes high variance. This is particularly problematic here because episodes are long and successful trajectories are sparse. Temporal-difference methods reduce variance through bootstrapping, but off-policy value-based learning was unstable in this project because the observation aliasing and heavy penalties produced noisy Q-targets. PPO offered a more stable compromise: direct policy optimization with a learned critic and generalized advantage estimation (GAE), which is itself a bias-variance tradeoff.

This argument mirrors the Monte Carlo versus TD discussion from Lectures 7 and 8 and the policy-gradient progression from Lectures 14 and 15. OBELIX needed something between high-variance return fitting and brittle short-horizon bootstrapping, and PPO with GAE was the most practical middle ground.

8.2 Asymmetric PPO

The main training idea was to use asymmetric actor-critic learning. The deployed actor receives only legal features x_t, while the critic receives both x_t and privileged simulator features p_t:

$$\pi_\theta(a_t \mid x_t), \qquad V_\psi(x_t, p_t).$$

This improves the value estimate during training without leaking privileged information into the final submission.

The privileged vector has dimension 34. It includes normalized robot position, sin / cos robot heading, box position, relative robot-box vector, relative distance, relative heading, push-active flag, visibility flag, stuck flag, step fraction, wall flag, difficulty, box speed, box velocity, boundary margins, goal distance, wall-gap geometry, and terms indicating which side of the wall the robot and box occupy. These inputs are used only by the training critic.

8.3 Objective Functions

The PPO actor objective uses the clipped surrogate

$$L_\pi(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta)\hat{A}_t, \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right],$$

where

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid x_t)}{\pi_{\theta_{\text{old}}}(a_t \mid x_t)}.$$

Advantages were computed with generalized advantage estimation:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V_\psi(x_{t+1}, p_{t+1}) - V_\psi(x_t, p_t).$$

The critic was trained with a clipped value loss and the actor with entropy regularization. The entropy term mattered in practice because otherwise the policy often collapsed into low-mobility turn-heavy behavior or overly aggressive repeated forward pushing.

8.4 Bias-Variance Interpretation

This training setup can be interpreted using the course discussion of MC versus TD. Pure Monte Carlo uses the full return and is unbiased but high variance; pure TD bootstraps from the critic and is lower variance but biased. PPO with GAE lies between these extremes. In this project that tradeoff was useful: the task needed lower-variance updates than pure return targets, but still required enough long-horizon signal to learn that early search and correct attachment geometry affect the final success bonus.

That is almost exactly how GAE was motivated in the actor-critic lectures: not as a mathematical embellishment, but as a practical way to get useful gradients in long horizon problems where neither pure Monte Carlo nor very short TD targets are ideal.

9 Final Model Architecture

9.1 Wall Branch

At inference time, the wall branch is an actor-only MLP:

$$271 \to 512 \to 256 \to 128 \to 5.$$

Hidden activations are tanh, and the output is a 5-dimensional logit vector over the five discrete actions. The checkpoint in the final submission contains both the actor weights and the full asymmetric training checkpoint for reproducibility, but only the actor backbone and policy head are loaded during deployment.

The wall branch was trained with `assym_ppo/train_asym_ppo.py`. The checkpoint metadata for the deployed wall actor records:

number of vectorized environments	512,
rollout length	128,
training steps	1,000,000,
actor hidden sizes	$[512, 256, 128]$,
critic hidden sizes	$[1024, 512, 256]$,
γ	0.995,
λ_{GAE}	0.95,
learning rate	1.5×10^{-4},
entropy coefficient	0.002,
warm-start expert	`eval300`,
warm-start episodes	20.

Reward shaping during this phase included visibility and IR bonuses, an enlarged stuck penalty, approach progress, alignment progress, and push progress. The policy was still screened using the environment reward, not only the shaped training reward.

9.2 No-Wall Branch

The no-wall branch is a standard PPO actor-critic checkpoint deployed as an actor MLP:

$$32 \to 128 \to 64 \to 5.$$

Its checkpoint in the final bundle is `ppo_lab/nowall_d3_ppo_v3.pth`. It was trained using `PPO/train_ppo.py` with the CUDA vectorized environment:

environment backend	`torch_vec`,
environment device	`cuda`,
number of environments	256,
rollout length	128,
total environment steps	4,194,304,
difficulty	3,
wall obstacles	`False`,
max steps	2000,
learning rate	2×10^{-4},
entropy coefficient	0.005,
update epochs	6,
initial checkpoint	`ppo_lab/nowall_d3_ppo_v1.pth`.

This branch was retained because it consistently produced much stronger no-wall returns than the recurrent and hand-written alternatives.

10 Mode Switching and Final Inference Policy

The final submission is not a single monolithic network. It is a lightweight mixture of two policies combined with an observation-only switch. This design arose from an empirical fact: the no-wall problem and the wall problem rewarded different behaviors, and a single model rarely handled both reliably.

This choice is consistent with a broader lesson from the lectures: representation and architecture should match problem structure. The course repeatedly emphasized that the agent design should reflect the state, action, and reward geometry of the environment, and in OBELIX the wall and no-wall regimes were simply too different to ignore.

For the first 80 steps, the agent runs the wall branch as a probe policy and records three summary statistics:

$$\begin{aligned} C &= \text{number of front-near/contact-like observations}, \\ B &= \text{number of blind observations}, \\ F &= \text{total number of front sensor hits}. \end{aligned}$$

At the end of the probe window, it switches to the no-wall branch when

$$C \leq 15, \qquad B \geq 40, \qquad F \geq 0.$$

Otherwise it stays with the wall branch. In practice, the first two inequalities are the real discriminators: no-wall starts often exhibit long blind-search segments with little early contact, whereas wall starts more often create early side/front contacts or stuck-like observations.

The final file also contains a small exact-pattern boost controller for two early observation patterns:

$$\begin{aligned} &(0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0), \\ &(1,1,0,1,0,0,0,0,0,0,0,0,1,1,1,1,0,0). \end{aligned}$$

This controller uses only observation bits, dead-reckoned heading from the agent's own turns, a small wall-hit memory, contact memory, and stuck recovery. It exists because a small number of starts repeatedly caused hesitation or early-contact failure in the neural branch.

Listing 1: High-level final policy logic.

```
def policy(obs, rng):
    if new_episode:
        reset_probe_and_memory()

    if initial_observation_matches_boost_pattern:
        if boost_is_still_active:
            return handmade_boost_controller(obs)

    if step < 80:
        update_probe_statistics(obs)
        return wall_policy(obs)

    if not decided:
        use_nowall = contact_count <= 15 and blind_count >= 40
        decided = True

    if use_nowall:
        return nowall_policy(obs)
    return wall_policy(obs)
```

10.1 Late Macro-Assisted and Distilled Variants

After the clean switch submission, I built later variants in `d3_wall_search/` to improve local weighted score and hidden-seed behavior. The most important one was `submission_d3_wall_macro_51.zip`. Its runtime policy still used the same legal interfaces: the 18-bit observation, memory from previous legal observations and actions, and `id(rng)` for episode reset. However, it added a macro layer before the normal switch:

1. match a small number of exact 18-bit observation signatures, such as all-zero starts and left-side starts;
2. take a one-step probe such as `R45` when the first observation was ambiguous;
3. if the next observation matched a known legal signature, play a fixed action sequence;
4. otherwise fall back to the exact-boost controller or the learned wall/no-wall switch.

This macro layer is important to describe honestly. It did not read simulator coordinates, rewards, `done`, `info`, wall flags, or hidden state. It was not illegal observation access in the runtime sense. But it was hand-coded policy knowledge extracted from local trajectory analysis. For that reason I treated the resulting 500+ local weighted score as policy overtraining rather than as evidence that the general wall problem was solved.

I also attempted to remove the hard-coded macro layer by distillation. The scripts `distill_macro_wall_student.py` and `distill_combo_students.py` collected expert trajectories from the macro wall policy and the strong no-wall policy, then trained recurrent students to imitate them. This produced `submission_d3_wall_student_balanced_v1.zip` and `submission_d3_learned_router_students_v1.zip`. The learned-router student reached very high local score, but hidden/-Codabench performance showed that it was still overfit. The lesson was that distillation can hide the explicit macro table, but it does not automatically create generalization if the demonstrations themselves cover only a small seed distribution.

11 Experimental Protocol

The clean switch submission package is `assym_ppo/submission_switch_d3_v3base/submission_switch_d3_v3base.zip`. The later macro-assisted and learned-router packages are in `d3_wall_search/`. Each submission package contains exactly `agent.py` and `weights.pth`. Local evaluation used `evaluate.py` with ten runs, base seed 0, difficulty 3, box speed 2, and $T = 2000$. Thus the evaluation seeds were $0, 1, \ldots, 9$.

The same protocol was used when screening strong candidates. This is important because OBELIX has high seed-to-seed variance. Comparing checkpoints under different horizons, difficulties, obstacle settings, or random seeds can give misleading conclusions. All comparisons reported below were therefore interpreted relative to the same local protocol whenever possible.

12 Final Evaluation and Candidate Selection

The local results for the clean switch submission are shown in Table 1. The no-wall branch is clearly strong: its mean return is highly positive and large enough that many seeds must have solved the task efficiently. The wall branch remains much harder and has high variance, but it was a useful baseline before the later macro/router experiments.

Condition	Mean return	Std. dev.	Runs
No wall, difficulty 3	1820.4	1539.7	10
Wall, difficulty 3	-1142.8	1846.5	10
Weighted score $0.4R_{\mathrm{nw}} + 0.6R_{\mathrm{w}}$	42.48	–	–

Table 1: Local evaluation of the clean switch submission `submission_switch_d3_v3base.zip`.

The large standard deviations show that this environment is not easy to summarize with a single mean. Later submissions achieved higher local scores by adding router distillation and macro logic, as shown in Table 2. These results must be interpreted carefully because local seed $0, \ldots, 9$ performance did not predict Codabench performance well.

Candidate	Wall	No-wall	Weighted	Comment
Clean switch baseline	-1142.8	1820.4	42.48	balanced local baseline
Macro-assisted wall policy	51.6	1252.9	532.1	local overfit
Learned-router student	51.6	1827.5	761.64	poor hidden transfer
Random-probe router	-865.7	1363.3	25.9	fixed random suite
Guarded conservative router	-1481.7	325.9	-758.7	broad proxy

Table 2: Late-stage candidate policies. The first three scores are local difficulty-3 evaluations on seeds $0, \ldots, 9$. The random-probe and guarded-router rows were evaluated on fixed proxy seed suites. Hidden/Codabench behavior was often worse than the local score, so these numbers are not a proof of general performance.

For comparison, several wall-only candidates screened after the final submission were worse on the same wall evaluation seeds.

Candidate	Wall mean return
Wall tuned v1	-1531.2
Wall tuned v2	-1531.2
Wall tuned v2 snapshot	-1531.2
Exact-boost rich warm-start	-1531.2
Asymmetric pose-memory wall	-80912.8

Table 3: Post-hoc wall checkpoint screen. These candidates were not selected because they did not improve the final wall branch.

13 Discussion and Interpretation

13.1 Why the Final Agent Worked Better

The final system improved over earlier attempts not simply because it used a larger network, but because its design matched the structure of the problem.

1. **It treated the task as partially observable.** Memory features were added only where they were needed, especially in the wall setting where short-term history can distinguish blind search, side contact, repeated stuck states, and approach phase.

2. **It separated actor information from critic information.** The privileged critic made value estimation easier during training, reducing variance and improving advantage quality, while the actor remained fully legal at inference time.

3. **It did not force a single behavior onto different regimes.** The no-wall and wall settings reward different local strategies. The switch policy allowed each branch to specialize without violating the deployment rules.

13.2 Why the Wall Setting Remained Hard

Even with the final design, the wall regime remained the bottleneck. This is expected. The wall setting combines all adverse factors at once: partial observability, moving or blinking targets, narrow geometry, and very high penalties for persistent contact. In such states, the policy must infer whether it is misaligned, whether the box is visible, and whether a forward move will improve progress or create another jam. A small error in local geometry can completely reverse the value of `FW`.

This is exactly the kind of setting where the course distinction between immediate reward and long-run value becomes important. A locally plausible forward move can be globally bad if it commits the robot to a stuck loop. Conversely, a short rotation sequence that looks unproductive in the short term may be necessary to obtain a high-value push angle.

13.3 Main Failure Modes Seen During Development

Across development, the main failure modes were:

- passive turn-heavy policies that avoided stuck penalties but never solved the task,

- aggressive push policies that attached from the wrong side and then accumulated repeated collision penalties,

- recurrent policies that looked strong under teacher mixing but collapsed when the teacher was removed,
- brittle hand-designed routines that failed under small changes in geometry or box motion.

The final submission is best understood as a direct response to these repeated empirical failure modes.

14 Reproducibility

The final evaluation can be reproduced with:

```
source /home/rycker/src/anaconda3/etc/profile.d/conda.sh
conda activate torch
cd "/home/rycker/study/CS780␣RL"

python CS780-OBELIX/evaluate.py \
  --agent_file CS780-OBELIX/assym_ppo/submission_switch_d3_v3base/agent.py \
  --agent_name switch_d3_v3base_nowall \
  --runs 10 \
  --seed 0 \
  --max_steps 2000 \
  --difficulty 3

python CS780-OBELIX/evaluate.py \
  --agent_file CS780-OBELIX/assym_ppo/submission_switch_d3_v3base/agent.py \
  --agent_name switch_d3_v3base_wall \
  --runs 10 \
  --seed 0 \
  --max_steps 2000 \
  --difficulty 3 \
  --wall_obstacles
```

The no-wall PPO branch was trained with a command equivalent to:

```
python CS780-OBELIX/PPO/train_ppo.py \
  --env_backend torch_vec \
  --env_device cuda \
  --device cuda \
  --out CS780-OBELIX/ppo_lab/nowall_d3_ppo_v3.pth \
  --init_checkpoint CS780-OBELIX/ppo_lab/nowall_d3_ppo_v1.pth \
  --num_envs 256 \
  --rollout_steps 128 \
  --total_env_steps 4194304 \
  --max_steps 2000 \
  --difficulty 3 \
  --box_speed 2 \
  --hidden_dims 128 64 \
  --lr 0.0002 \
  --ent_coef 0.005 \
  --update_epochs 6 \
  --minibatch_size 8192 \
  --schedule fixed \
  --fw_bias_init 1.0 \
```

```
  --rec_enco \
  --seed 41
```

The wall actor branch was trained with asymmetric PPO, using only legal actor features for the policy and privileged features for the critic:

```
python CS780-OBELIX/assym_ppo/train_asym_ppo.py \
  --out CS780-OBELIX/assym_ppo/wall_tuned_v1.pth \
  --env_device cuda \
  --device cuda \
  --num_envs 512 \
  --rollout_steps 128 \
  --total_env_steps 1000000 \
  --max_steps 300 \
  --difficulty 0 \
  --wall_obstacles \
  --obs_stack 12 \
  --action_hist 6 \
  --actor_hidden_dims 512 256 128 \
  --critic_hidden_dims 1024 512 256 \
  --lr 0.00015 \
  --ent_coef 0.002 \
  --reward_scale 5.0 \
  --reward_clip 100.0 \
  --visible_bonus 0.5 \
  --ir_bonus 1.5 \
  --stuck_extra_penalty 2.0 \
  --approach_progress_bonus 35.0 \
  --alignment_bonus 0.75 \
  --push_progress_bonus 120.0 \
  --warm_start_expert eval300 \
  --warm_start_episodes 20 \
  --warm_start_epochs 4 \
  --eval_runs 10 \
  --seed 0
```

15 Legality and Deployment

The final submitted policy reads only the observation argument passed to `policy(obs, rng)`. The privileged critic and simulator-state tensors are strictly training-only.

At inference time:

- the wall actor receives only x_t, derived from legal observations and past actions,
- the no-wall actor receives only the current observation through the 32-dimensional legal encoder,
- the switch uses only early observation counts,
- the handcrafted boost uses only observation bits and dead reckoning from the policy's own actions.

The code uses the evaluator's `rng` object as an episode-reset key for stateful memory, but it does not sample random actions or inspect simulator state. If a stricter interpretation forbids using `rng`

identity as an episode-boundary signal, the observation-reset submission in `strict_obs/` should be used instead, although it scored lower locally.

16 Conclusion

The main reason the final agent outperformed the earlier attempts was not raw model size alone. The successful design fixed the main structural weaknesses encountered throughout the project:

1. it modeled OBELIX correctly as a partially observable control problem rather than a purely reactive sensor-action map,
2. it used asymmetric PPO so that value estimation could exploit privileged training information while the actor remained fully deployable,
3. it specialized the no-wall behavior separately from the wall behavior,
4. it used a simple legal switch to route early trajectories to the more appropriate expert branch.

This combination avoided the most common failure modes seen during the search: passive turning policies, wrong-side wall attachments, brittle hand-written recovery, and teacher-dependent recurrent collapse. The final weighted score is still limited by the hard wall regime, but the resulting submission is the strongest balanced controller found under the available compute budget and the project rules.